

2026.05.08

SPI & I2C + AXI 설계 및 UVM 검증

김수빈

Contents

01

개요

02

AXI

03

SPI & SPI UVM

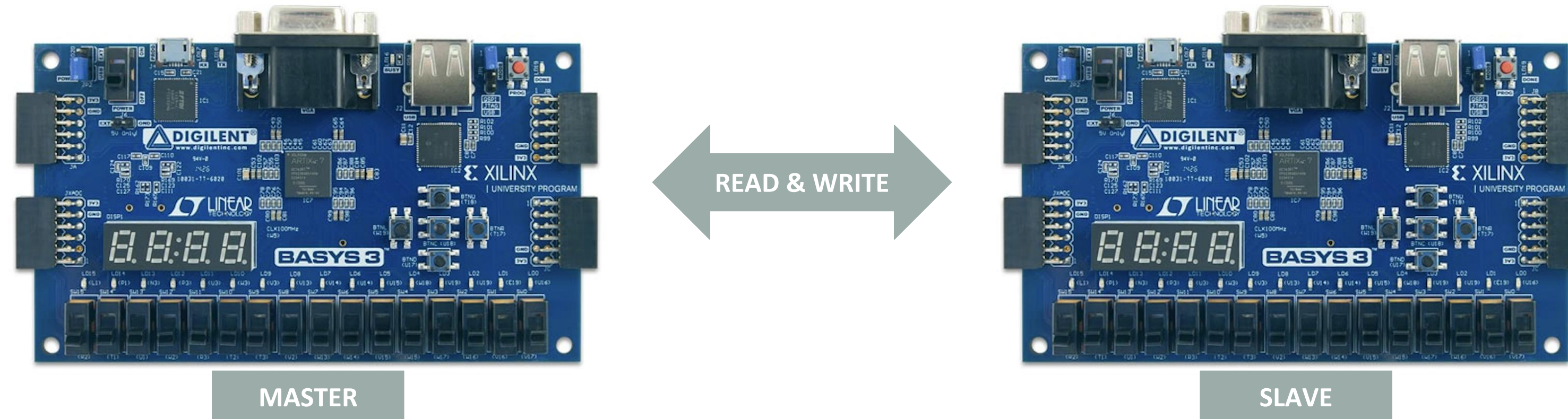
04

I2C

05

Troubleshooting & 느낀점

Overview 개요



- AXI 기반 SPI / I2C Master IP 설계
 - Vivado Block Design 구성
 - C 코드(Microblaze)로 레지스터에 Write / Read 로 동작 제어
 - 2개의 Board로 Write / Read 통신 구현
-
- AXI4 -Lite 사용 이유
 - CPU가 IP를 메모리 주소처럼 제어 표준 인터페이스로 연결

AXI



Point-to-Point Interface

01

AXI4

AXI : Advanced eXtensible Interface

AXI

- ARM AMBA 표준 – SoC / FPGA 에서 IP 블록을 연결하는 인터페이스
- Bus 가 아닌 Point-to-Point 전용선 → Chip Select 없이 1:1 연결
- 5개 독립 채널: AW · W · B · AR · R (Write/Read 동시 가능)

AXI4-Lite

- AXI4 의 경량 버전 – 레지스터 제어에 특화
- 한 번에 1개 데이터 전송 (burst 없음) → 구현 단순
- CPU 가 메모리 주소에 write/read 하듯 IP 레지스터 제어
- Xilinx IP (SPI·I2C Master) 가 AXI4-Lite 인터페이스 지원

AXI4 vs APB

AXI는 Bus가 아닌 Point-to-Point Interface → Signal과 Timing만 맞추면 됨

APB / AHB Bus

- Broadcasting으로 데이터 전송
- Decoder + Mux 필요
- Chip Select 개념 있음

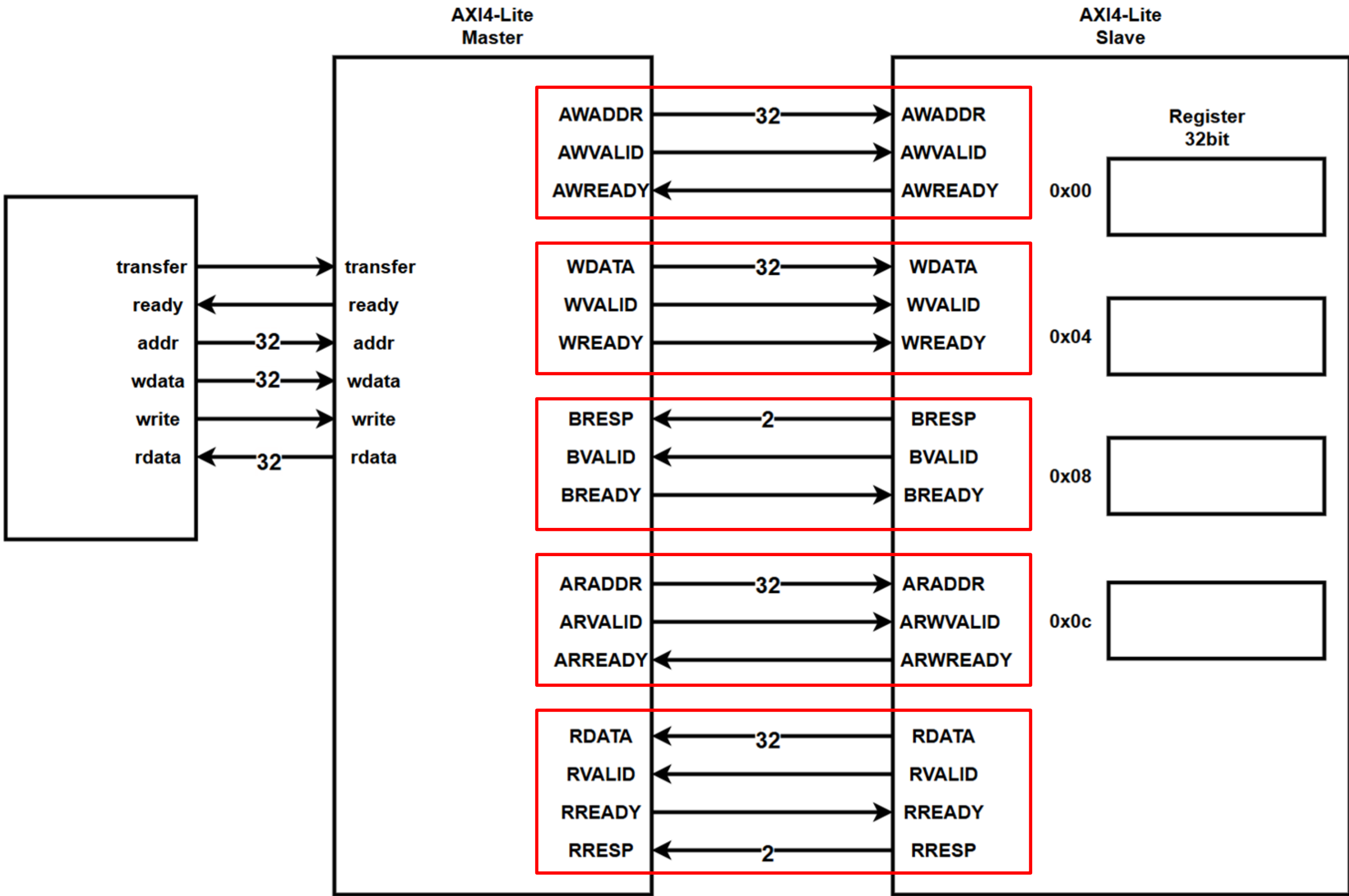
VS

AXI Interface

- Point-to-Point 전용선 연결
- Interconnect가 라우팅 담당
- Chip Select 개념 없음

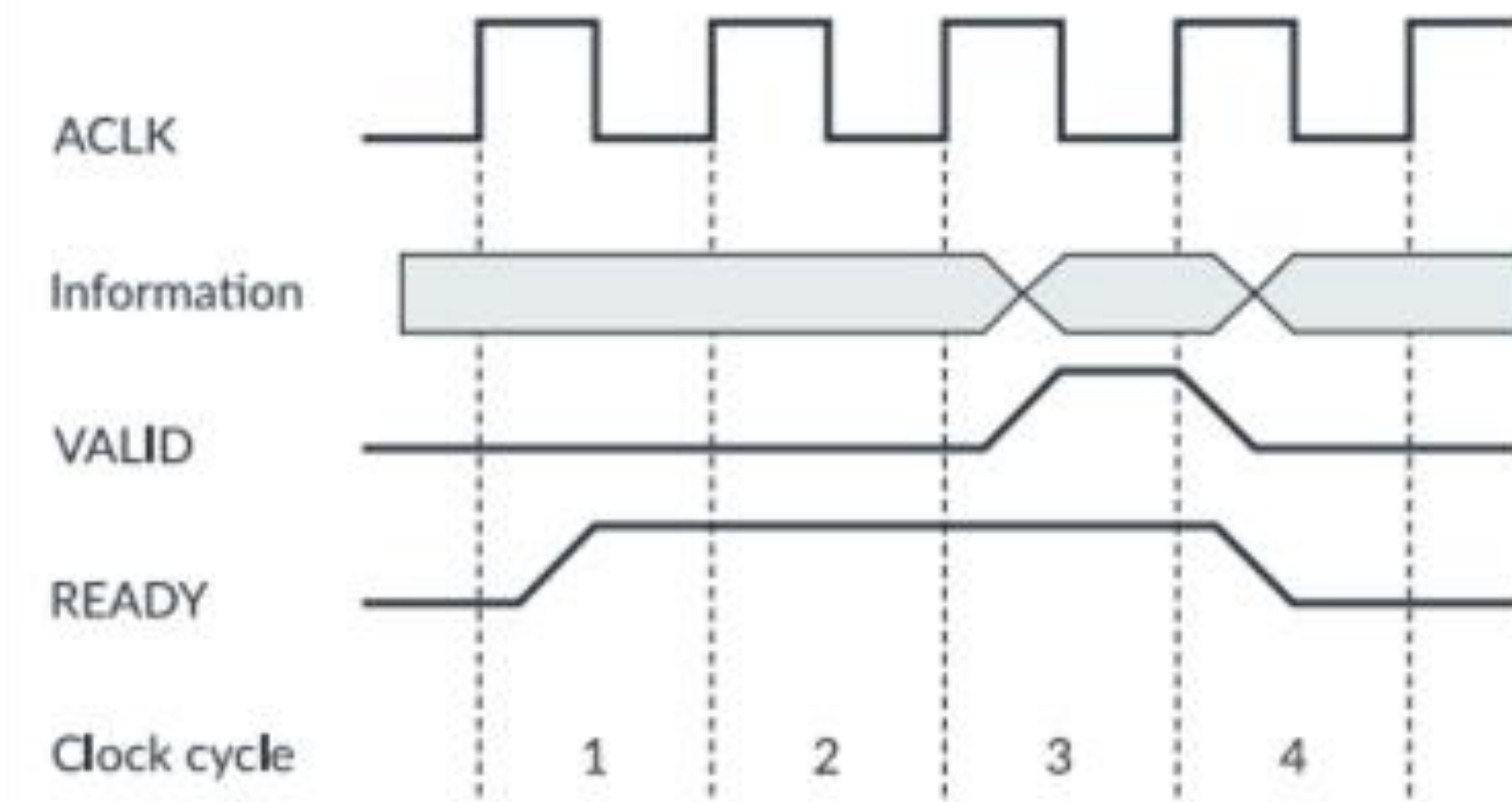
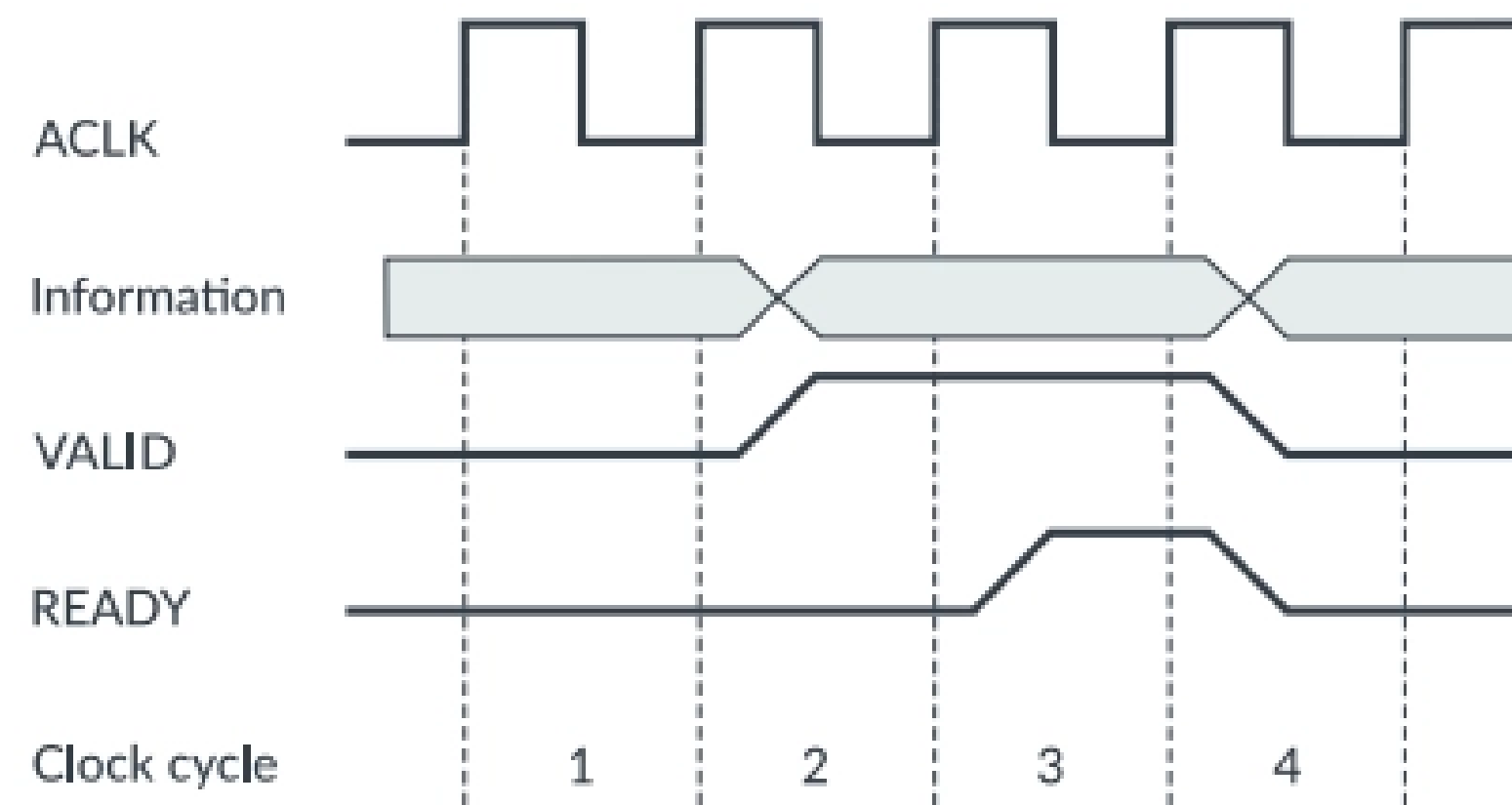
AXI4 Block Diagram

5개의 채널, 각각 독립적인 전용 선을 가지고 있음



AXI4 Handshake

Valid & Ready 동시 → High Clock Rising Edge에서 전송 완료



VALID

- 정보가 유효함을 알림
- Handshake 전까지 High 유지

READY

- 수신 준비 완료를 알림
- Ready를 미리 올리면 1클럭에 처리 가능

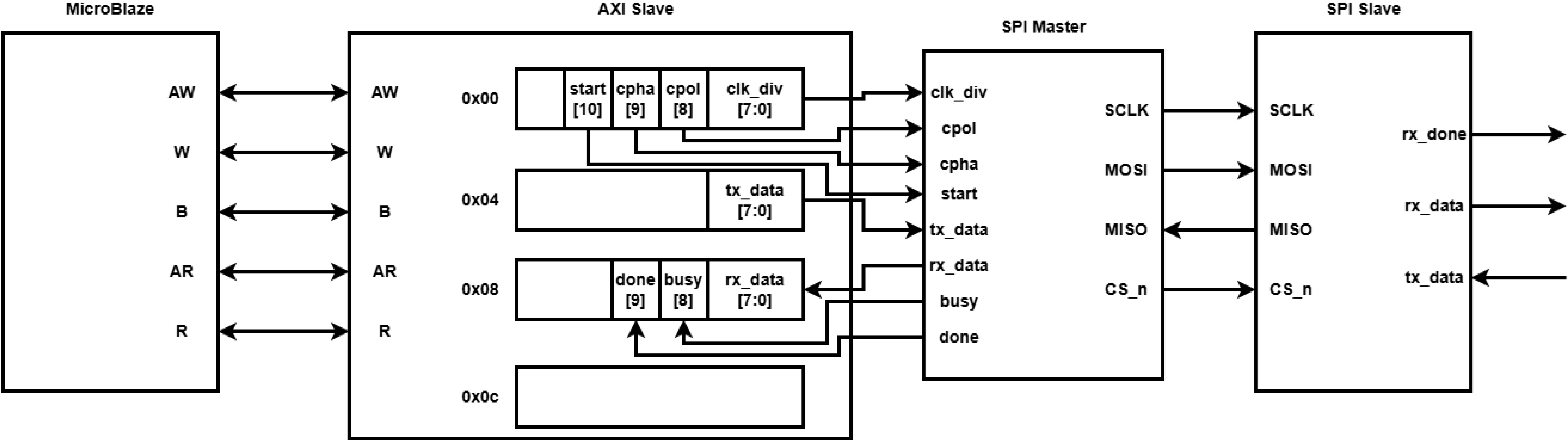
SPI



Master가 생성한 클럭을 기준으로 데이터를 동시에 송수신하는
고속 Full Duplex 통신

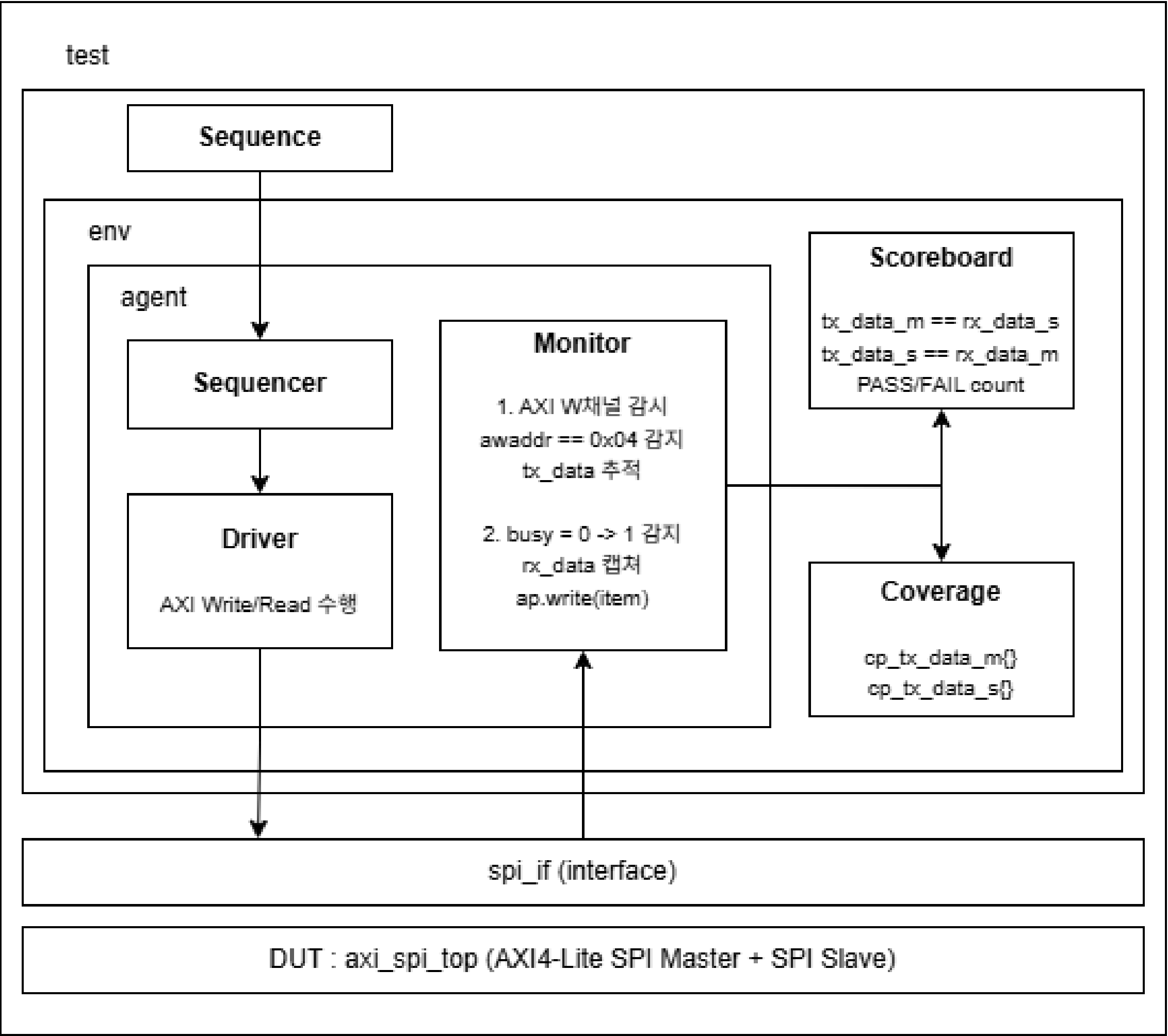
02

SPI + AXI Block Diagram



SPI UVM

Test Bench



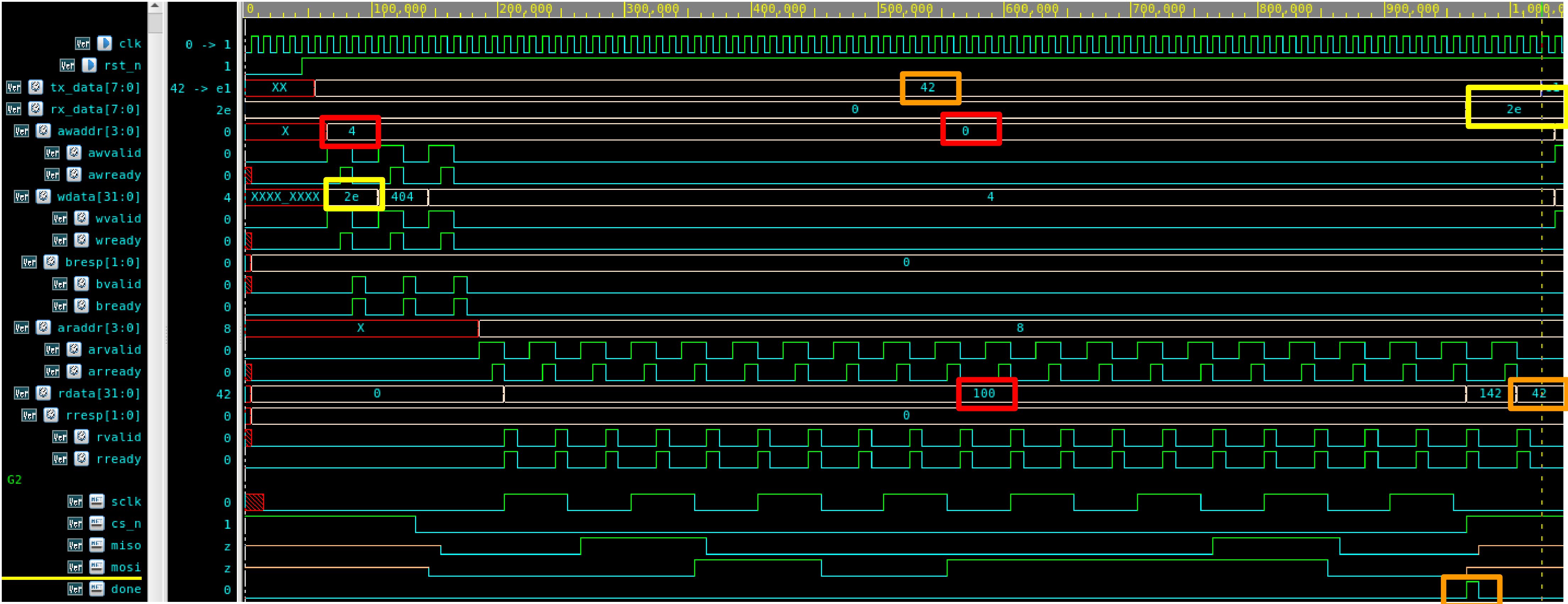
검증 시나리오 (1)

Write 채널로 rx 레지스터에 기록한 값
==
SPI 통신을 통해 Slave에 수신된 값

검증 시나리오 (2)

Slave가 MISO로 송신한 값
==
SPI 통신을 통해 Read 채널로 수신된 값

SPI UVM Simulation



```
==== Coverage Summary ====  
Overall : 100.0%  
tx_data_m : 100.0%  
tx_data_s : 100.0%  
==== Coverage Summary ====
```

```
[spi_scoreboard] Result : ** PASS **  
[SCB] PASS: 2560  
[SCB] FAIL: 0  
[spi_scoreboard] *****
```

SPI UVM Coverage

Total Transaction
2,560

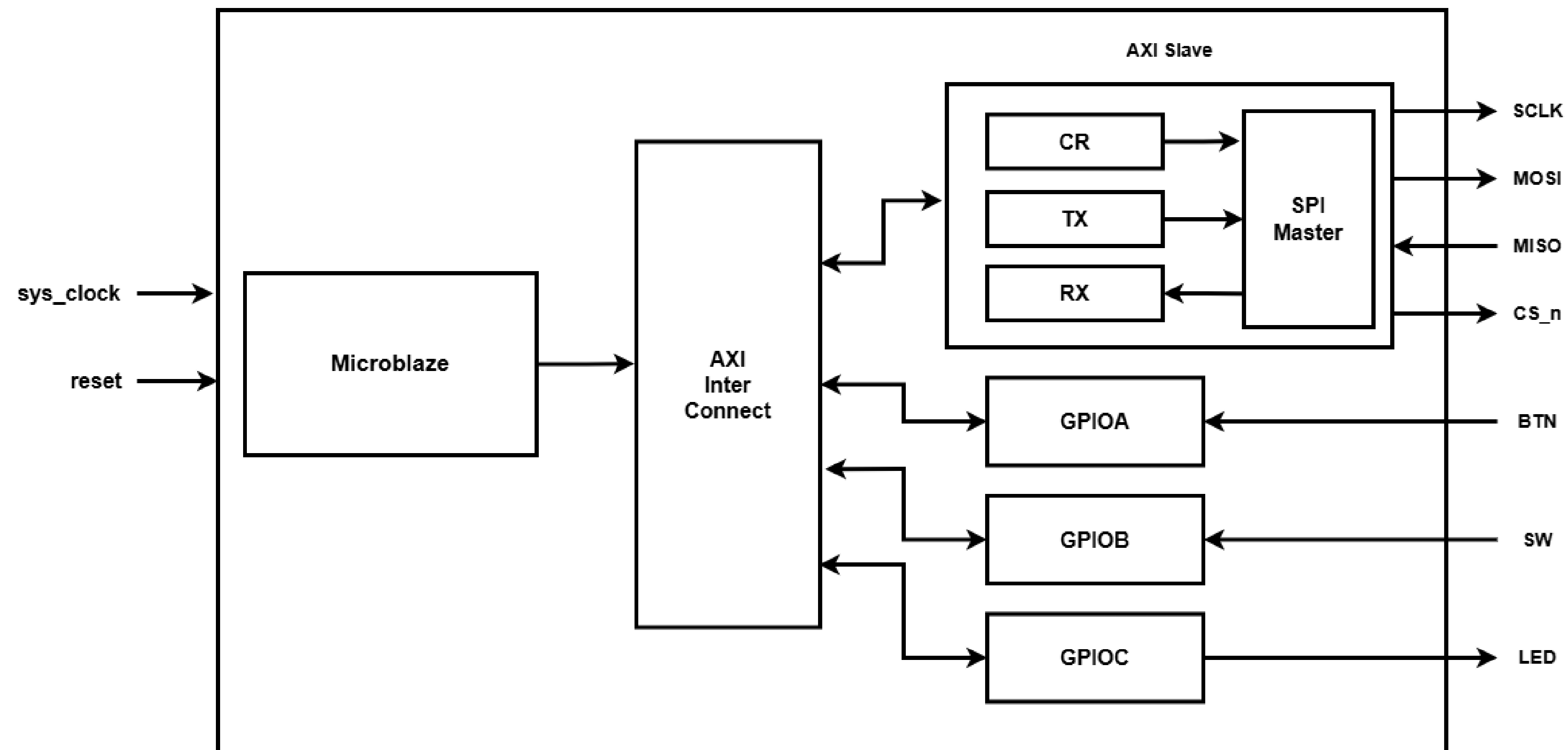
Bin Per Coverpoint
21

Bin Per Coverpoint
0x55
0xAA
0x01
0x80

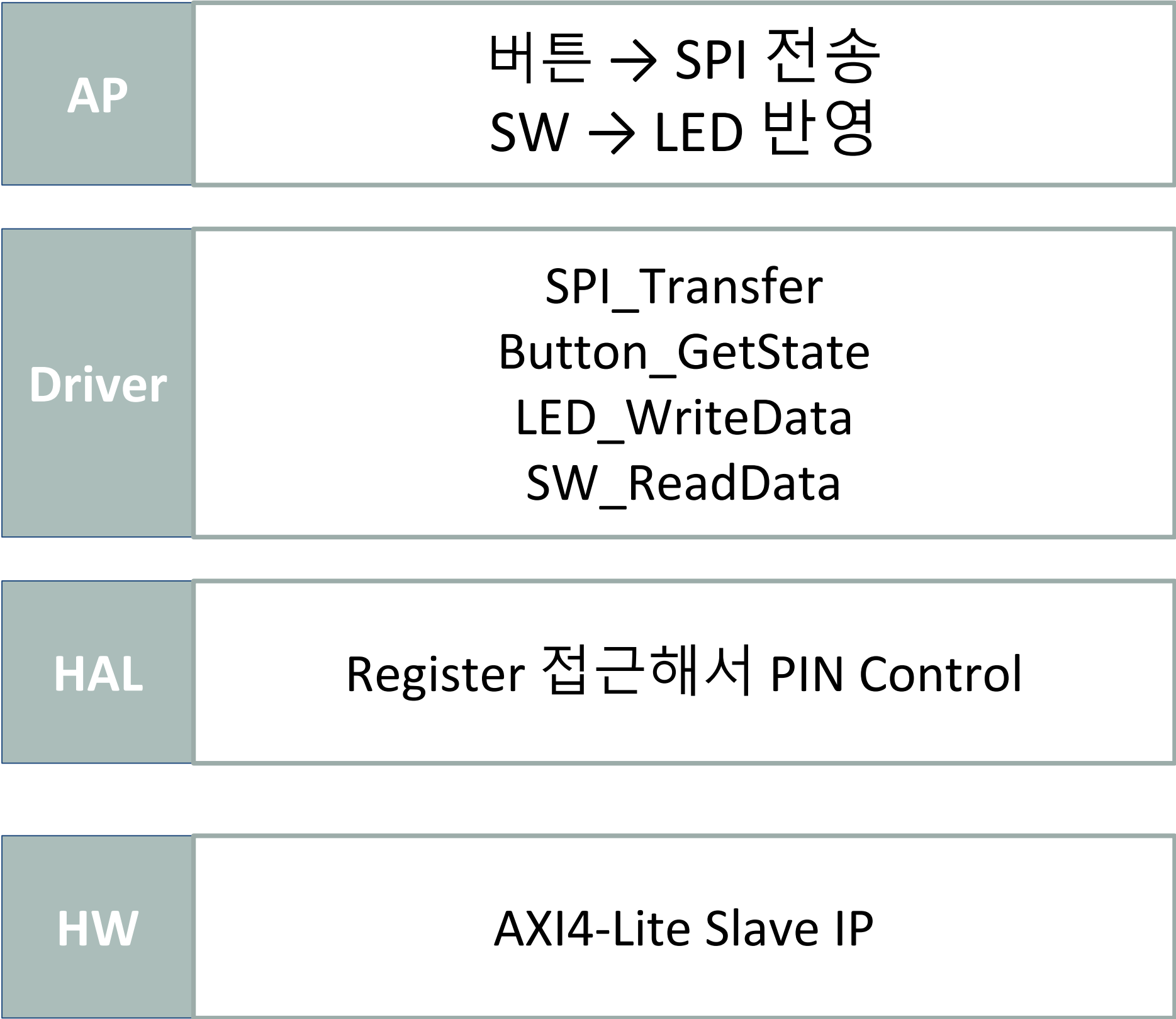
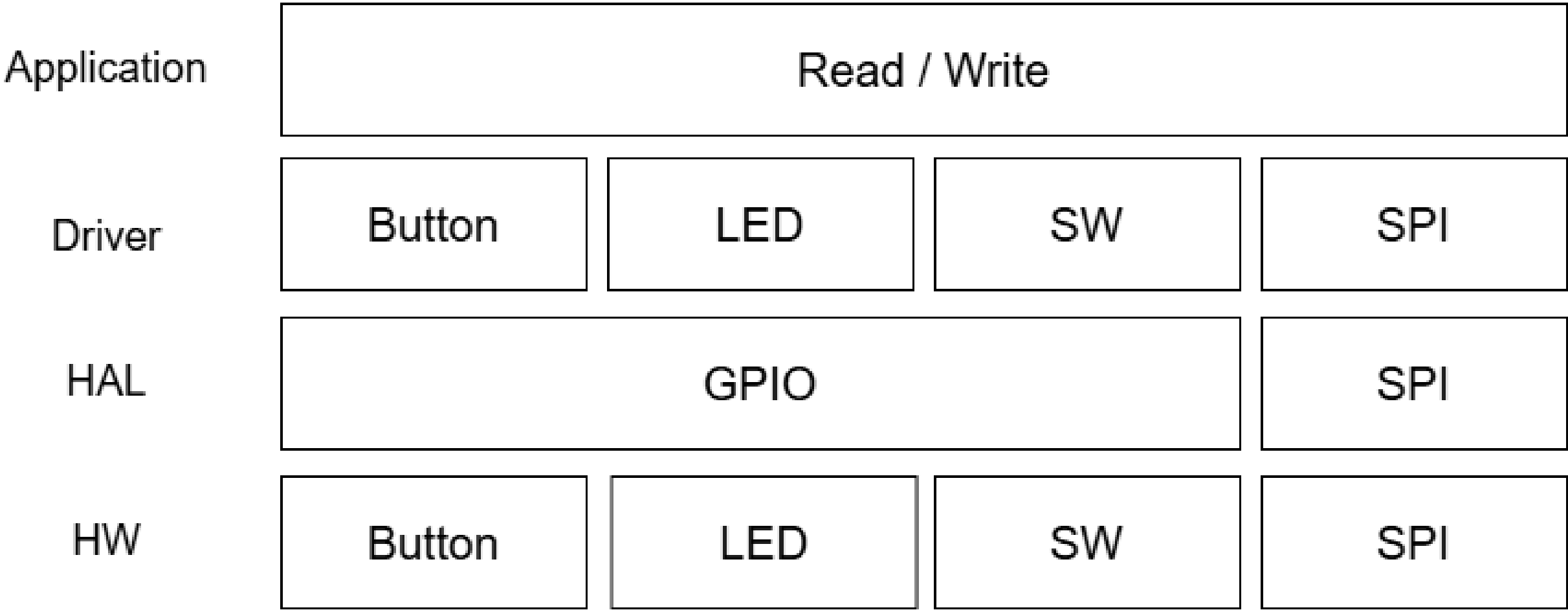
cp_tx_data_m						
Status	Bin Name	Type	At Least	Size	Hit Count	
✓	alt_01	User	1	1	8	
✓	alt_10	User	1	1	8	
✓	lsb_only	User	1	1	11	
✓	msb_only	User	1	1	15	
✓	range0	User	1	1	159	
✓	range1	User	1	1	143	
✓	range2	User	1	1	156	
✓	range3	User	1	1	161	
✓	range4	User	1	1	165	
✓	range5	User	1	1	141	
✓	range6	User	1	1	183	
✓	range7	User	1	1	169	
✓	range8	User	1	1	169	
✓	range9	User	1	1	170	
✓	rangea	User	1	1	156	
✓	rangeb	User	1	1	167	
✓	rangec	User	1	1	146	
✓	ranged	User	1	1	182	
✓	rangee	User	1	1	146	
✓	rangef	User	1	1	147	
✓	zero	User	1	1	10	

cp_tx_data_s						
Status	Bin Name	Type	At Least	Size	Hit Count	
✓	alt_01	User	1	1	8	
✓	alt_10	User	1	1	8	
✓	lsb_only	User	1	1	11	
✓	msb_only	User	1	1	15	
✓	range0	User	1	1	159	
✓	range1	User	1	1	143	
✓	range2	User	1	1	156	
✓	range3	User	1	1	161	
✓	range4	User	1	1	165	
✓	range5	User	1	1	141	
✓	range6	User	1	1	183	
✓	range7	User	1	1	169	
✓	range8	User	1	1	169	
✓	range9	User	1	1	170	
✓	rangea	User	1	1	156	
✓	rangeb	User	1	1	167	
✓	rangec	User	1	1	146	
✓	ranged	User	1	1	182	
✓	rangee	User	1	1	146	
✓	rangef	User	1	1	147	
✓	zero	User	1	1	10	

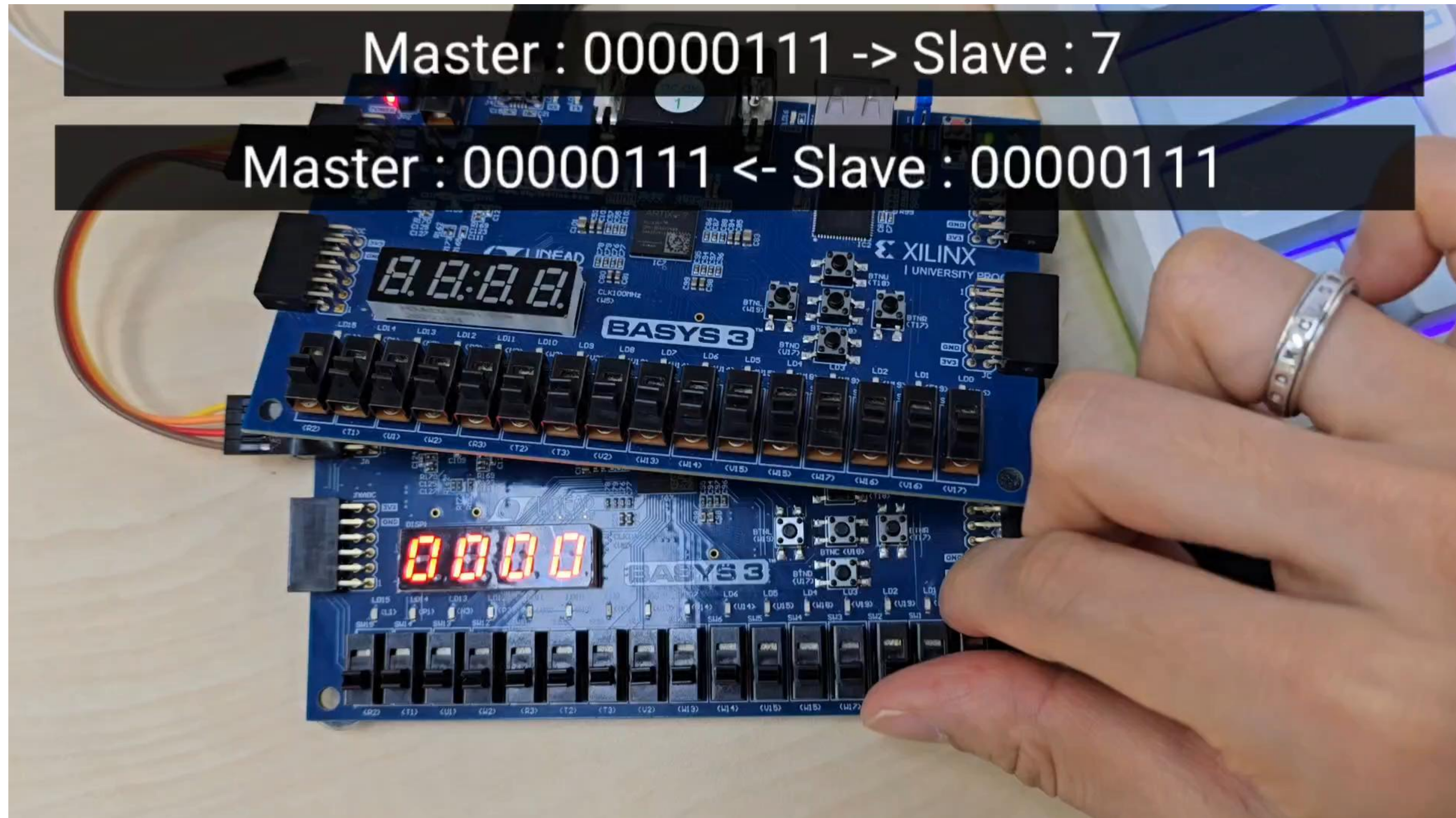
Microblaze + AXI + SPI + GPIO Block Diagram



SPI + AXI Stack Architecture



SPI 동작 영상



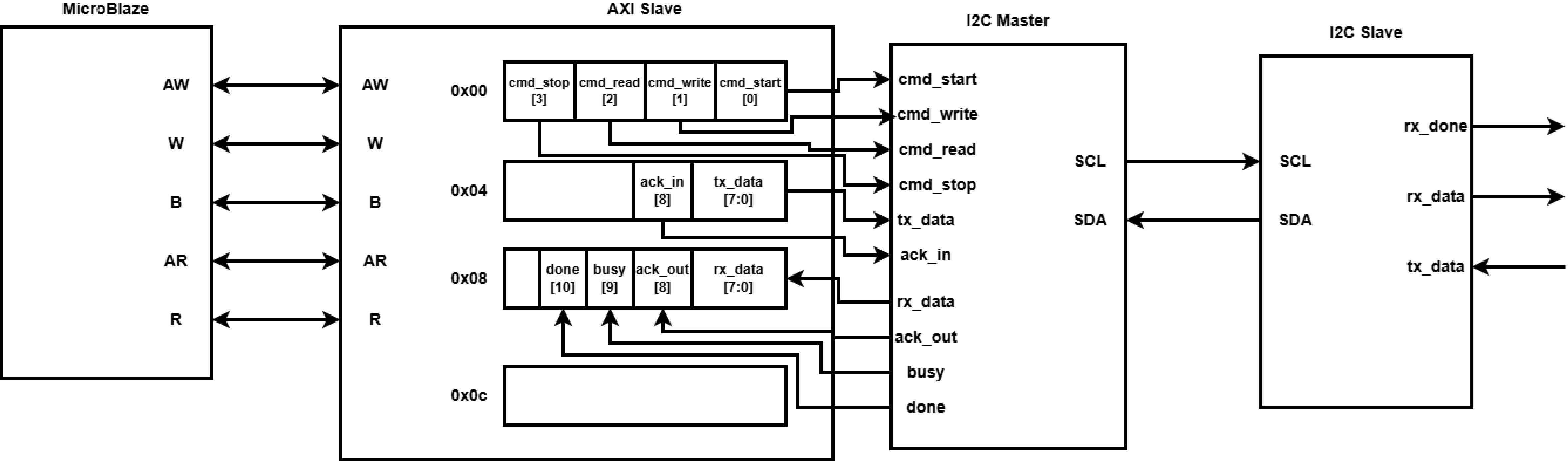
I2C



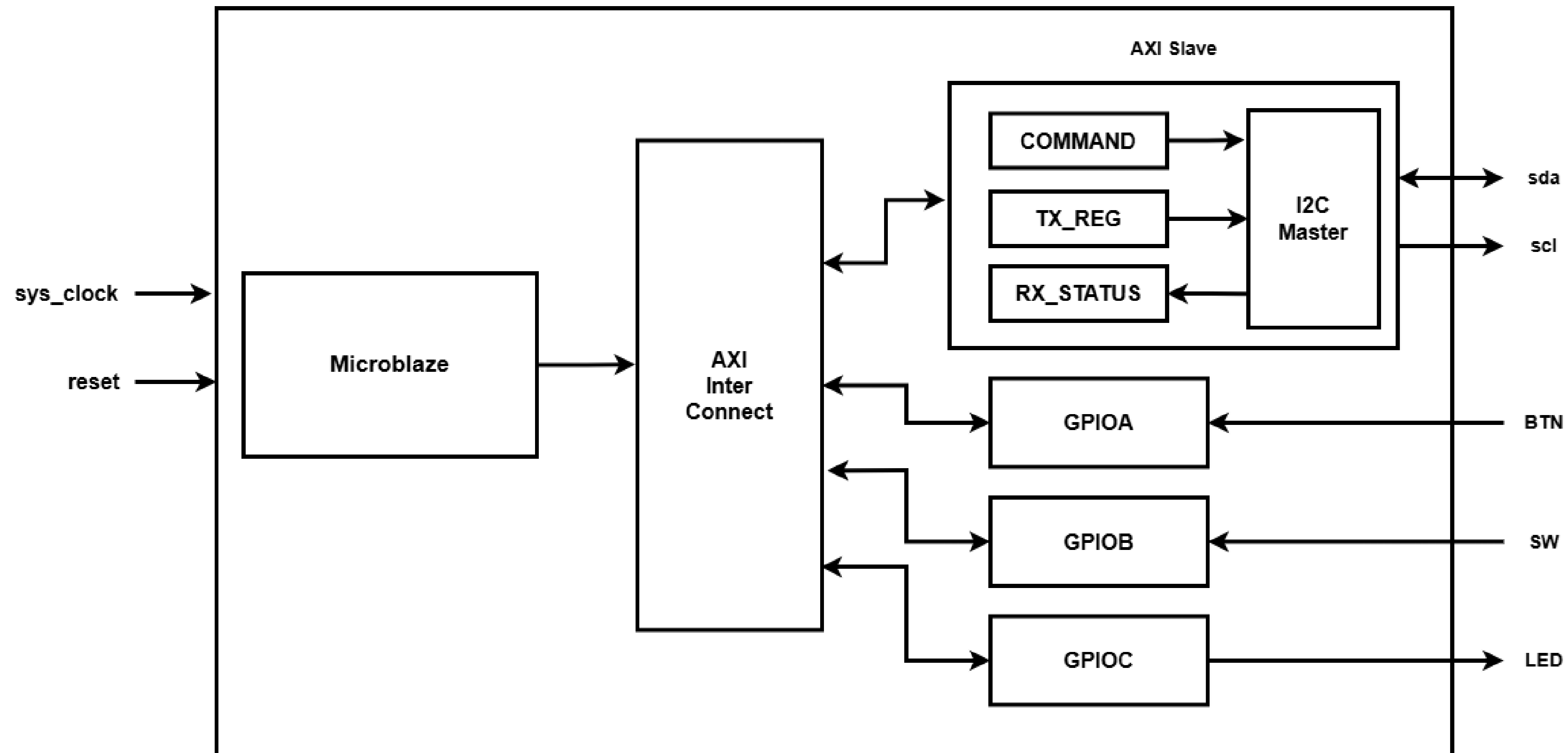
두 개의 선(SCL, SDA)을 사용하여 여러 디바이스가 주소 기반으로 통신하
는
Half Duplex 방식

03

I2C + AXI Block Diagram



Microblaze + AXI + I2C + GPIO Block Diagram

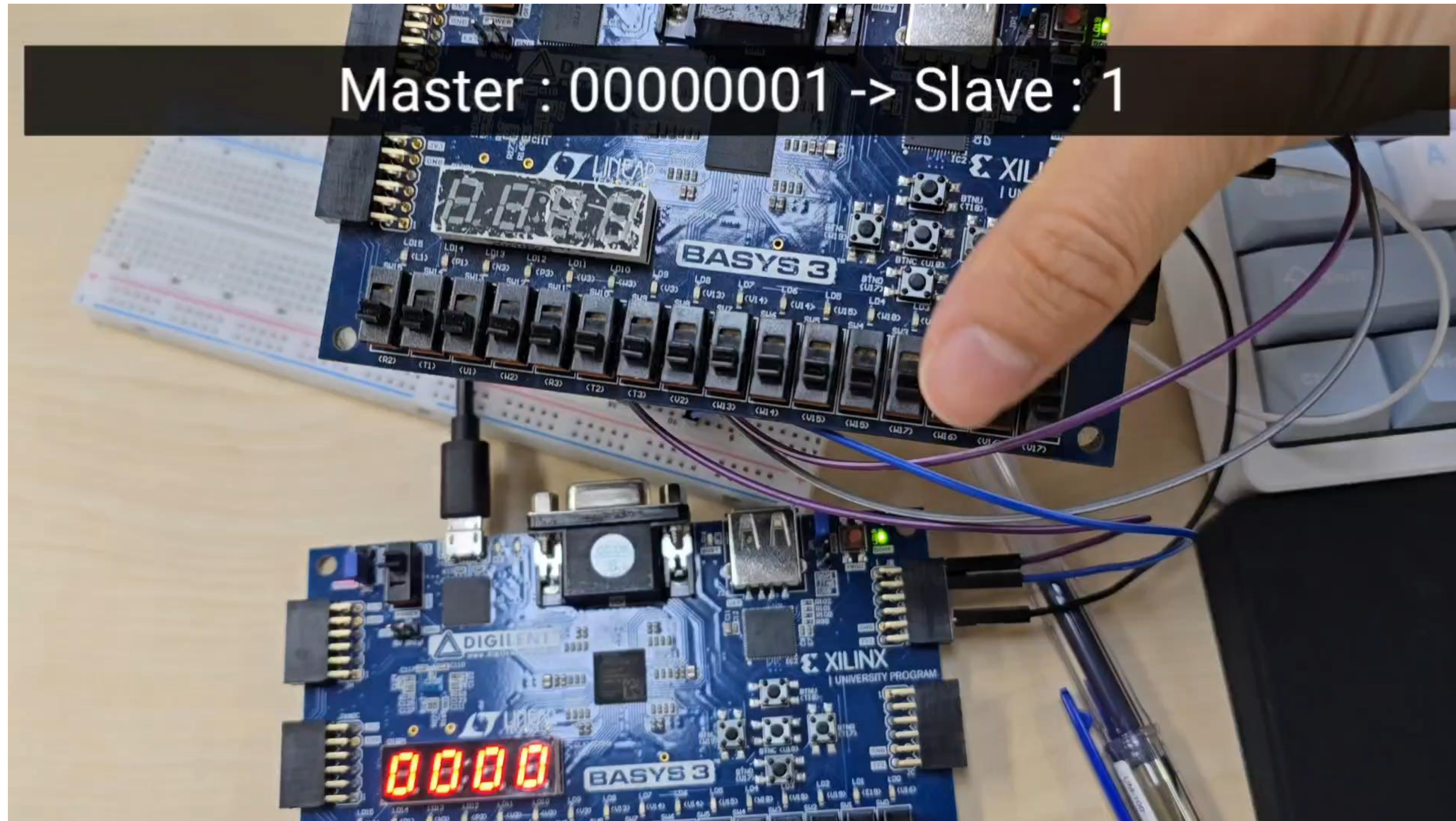


I2C + AXI Stack Architecture

Application	Read / Write			
Driver	Button	LED	SW	I2C
HAL	GPIO			I2C
HW	Button	LED	SW	I2C

AP	버튼 → I2C 전송 SW → LED 반영
Driver	I2C_Write_Execute / I2C_Read_Execute Button_GetState LED_WriteData SW_ReadData
HAL	Register 접근해서 PIN Control Command 신호 / Wait_Done
HW	AXI4-Lite Slave IP

I2C 동작 영상



Troubleshooting 및 느낀점

04

TroubleShooting : I2C Done 신호 잡기

명령어 처리가 끝난 후에 다음 명령어 처리 → Done 신호를 기다림

SPI

Done 신호 대신 Busy 신호로 판단

```
SPI_Start(SPIx, 1);  
while (SPI_GetRX_Busy(SPIx) == 0);  
//while (SPI_GetRX_Done(SPIx) == 1);
```

```
SPI_Start(SPIx, 1);  
while (SPI_GetRX_Busy(SPIx) == 0);  
//while (SPI_GetRX_Done(SPIx) == 1);
```

I2C

IP에서 Done 신호 다음 명령어까지 유지

```
reg done_real;  
  
always @(posedge S_AXI_ACLK) begin  
    if (S_AXI_ARESETN == 1'b0) begin  
        done_real <= 1'b0;  
    end else begin  
        if (done) begin  
            done_real <= 1'b1;  
        end else if (slv_reg_wren && (axi_awaddr[ADDR_LSB+OPT_MEM_ADDR_BITS:ADDR_LSB] == 2'h0)) begin  
            done_real <= 1'b0;  
        end  
    end  
end  
end
```

```
void I2C_CMD_START(I2C_TypeDef_t *I2Cx) {  
    I2Cx->COMMAND = (1 << I2C_CMD_START_BIT);  
    I2C_Wait_Done(I2Cx);  
    I2Cx->COMMAND = 0x00; // done 확인 후 클리어  
}
```

느낀점

- 디버깅을 위해 넣은 UART printf가 SPI 타이밍을 밀리게 해서 오히려 오류를 늘렸고, 관측 행위 자체가 시스템에 영향을 준다는 것을 다시 한 번 인지할 수 있었습니다.
- c언어 구조체가 아직 익숙하지 않지만, 하드웨어를 제어하는 코드에서 구조체가 왜 필수인지 이번 프로젝트를 통해 이해했고, 반복 학습을 통해 자연스럽게 사용할 수 있도록 연습해야겠다고 느꼈습니다.
- 같은 조원분들과 모르는 것을 물어보고 답하는 과정에서 제가 놓치고 있던 부분들도 익힐 수 있었습니다.

감사합니다.